

PLATFORM DOCUMENTATION

7ALP's

A complete walkthrough of the storefront, admin console, and B2B wholesale portal — every login, route, and feature, as currently implemented.

Updated 2026-07-16 · Frontend: React 19 + Vite + Redux Toolkit + Tailwind 4 · Backend: Node.js + Express 5 + MongoDB (Mongoose)

1. Architecture Overview

The platform is a two-app monorepo: a React single-page app (`frontend/`) talking to a REST API (`backend/`) mounted under `/api/v1` . There are three independent authentication systems — Customer, Admin, and B2B — each with its own JWT stored in an httpOnly cookie, so a login in one role never grants access in another.

Frontend stack

- React 19, React Router 7
- Redux Toolkit (cart, auth, B2B draft-quote state)
- Tailwind CSS 4
- Axios (shared instance, cookie credentials on)
- Google Maps JS API (Places + Geocoding)
- Razorpay Checkout.js

Backend stack

- Express 5, MongoDB via Mongoose
- JWT + httpOnly cookies per role
- Cloudflare R2 for product images
- Razorpay Node SDK
- Helmet, rate limiting, custom sanitize middleware

2. Logins

Customer

Mobile number + OTP, no password. A single flow handles both signup and login — if the number isn't recognized, the customer is asked for a name and an account is created automatically on OTP verification.

Field	Detail
Login route	<code>/customer/login</code> → <code>/customer/verify-otp</code>
Session	JWT in httpOnly cookie <code>customerToken</code>
Dev fallback OTP	<code>123456</code> always works until a real SMS provider is wired up

Admin

Email + password. Two roles: `Admin` and `SuperAdmin` (currently identical permissions — the distinction exists in the schema for future use).

Field	Detail
Login route	<code>/admin/login</code>
Session	JWT in httpOnly cookie <code>adminToken</code>

Seeded account	<code>7alpsadmin@7alps.com</code> / <code>7alps123</code> (run <code>npm run seed</code> in <code>backend/</code> ... see note below)
----------------	---

Careful with `npm run seed` . That script reseeds the *product catalog* from scratch (it deletes existing products first) — it is not the admin-account seeder. The admin account is seeded via `node dev-data/seedAdmin.js` directly, which is safe to re-run (it checks before creating).

B2B (marketing team)

Email + password, same shape as Admin login, but a completely separate account collection. B2B accounts are **created by an admin** — there is no self-signup. Each B2B login represents an internal marketing-team member who sells in bulk on behalf of a business buyer.

Field	Detail
Login route	<code>/b2b/login</code>
Session	JWT in httpOnly cookie <code>b2bToken</code>
Account creation	Admin → B2B Team → Add member
Seeded account	<code>b2bpartner@7alps.com</code> / <code>7alps123</code> (<code>npm run seed:b2b</code>)

3. Route Map

Public (no login)

Route	Page
<code>/</code>	Home — hero, categories, featured products, process, testimonials, FAQs
<code>/products</code>	Full catalog with filters
<code>/products/:id</code>	Product detail — gallery, info, ingredients, reviews
<code>/our-process</code>	Sourcing/production story
<code>/why-7alps</code>	Brand positioning / comparison
<code>/global-trade</code>	Export/trade information
<code>/partners</code>	Partnerships page
<code>/b2b-portal</code>	Wholesale marketing/landing page (static content, not the logged-in B2B portal)
<code>/contact</code>	Contact form + map embed
<code>/cart</code>	Shopping cart (guest or logged-in)

Customer (requires customer login)

Route	Page
<code>/checkout</code>	Shipping address (with map picker) + payment method + place order
<code>/customer/orders</code>	Order history
<code>/customer/profile</code>	Personal details + address book (with map picker)

Admin (requires admin login)

Route	Page
<code>/admin</code>	Dashboard — customers, orders, products, revenue
<code>/admin/products</code>	Product CRUD, variants, image upload
<code>/admin/categories</code>	Category CRUD
<code>/admin/orders</code>	All orders (customer + B2B), status updates
<code>/admin/customers</code>	Customer list with order/spend stats
<code>/admin/reviews</code>	Review moderation
<code>/admin/b2b</code>	B2B Team — members + quote approval

B2B (requires B2B login)

Route	Page
<code>/b2b</code>	Dashboard — quotes, orders, sales stats
<code>/b2b/catalog</code>	Browse products, propose bulk price/qty
<code>/b2b/request-quote</code>	Buyer details (with map picker) + submit + quote history
<code>/b2b/orders</code>	Orders created from approved quotes

4. Feature Walkthrough

Customer-facing storefront

- **Browse & filter** — category, price, search, rating sort.
- **Cart** — guest cart in localStorage; merges into the server cart automatically on login.
- **Checkout** — shipping address form with an embedded Google Maps picker (search or drag a pin to auto-fill address fields); choice of **Cash on Delivery** or **Pay Online (Razorpay)**.
- **Orders** — history with status, itemized breakdown, shipping address.
- **Profile** — editable name/email; a real address book (add/edit/delete/set-default), each entry fillable via the same map picker.

- **Reviews** — logged-in customers can rate & review a product they're viewing; reviews are held as **Pending** until an admin publishes them.

Admin console

- **Dashboard** — live counts: customers, orders, products, revenue.
- **Products** — full CRUD, multiple variants per product (label/price/MRP/stock), multi-image upload to Cloudflare R2.
- **Categories** — add/edit/activate/delete; delete is blocked if products still reference the category, with a clear error explaining why.
- **Orders** — every order (storefront and B2B) in one place; inline status changes (Confirmed → Processing → Shipped → Delivered/Cancelled).
- **Customers** — every registered customer with their order count and lifetime spend.
- **Reviews** — moderate: publish, flag, or delete any submitted review.
- **B2B Team** — create marketing-team logins; see each member's total bulk orders and sales; review a pending quote (adjust quantity/price if needed), approve it — which atomically decrements stock and creates a real order — or reject it with a reason.

B2B wholesale portal

- **Catalog** — the same live product catalog; the member picks a variant, a bulk quantity (10+ units), and proposes their own discounted unit price (capped at the listed retail price).
- **Request Quote** — the products picked in Catalog collect into a draft; the member adds the buyer's business name and delivery address (map picker included) and submits it for admin approval. Past quotes and their status are listed below the form.
- **Orders** — once admin approves a quote, it becomes a real order here.
- **Dashboard** — quote counts by status, total orders, total sales.

5. Payments

Cash on Delivery is the default and always available. **Razorpay is fully live and has been verified with a real completed test transaction** — order creation, the checkout modal, signature verification, and order confirmation all work end to end. If `RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET` in `backend/.env` are ever blanked out, attempting to pay online returns a clear "not configured yet" error rather than failing silently — it never crashes the server either way.

How it works: the backend computes the charge amount itself from the customer's live server-side cart — it never trusts a client-supplied amount, which is the standard payment-security practice (a client could otherwise tamper with what it pays). `POST /customer/payments/razorpay-order` creates the Razorpay order sized to that amount; the frontend opens Razorpay's Standard Checkout modal with it; on success, `POST /customer/payments/razorpay-verify` checks the HMAC-SHA256 signature Razorpay returns before creating the real order, decrementing stock, and clearing the cart — mirroring the same transaction the COD path uses.

6. Testing Reference

What	Value
Admin login	<code>7alpsadmin@7alps.com</code> / <code>7alps123</code>
B2B login	<code>b2bpartner@7alps.com</code> / <code>7alps123</code>
Customer login	Any 10-digit mobile number, OTP <code>123456</code>
Razorpay test card (recommended)	<code>5267 3181 8797 5449</code> (Mastercard) — any future expiry, any 3-digit CVV
Razorpay test OTP	Any 4-10 digit number (e.g. <code>1234</code>) — it's a test-mode simulation, not a fixed code

Avoid the generic `4111 1111 1111 1111` Visa test card — Razorpay's test environment treats it as a foreign-issued card, and most test accounts (this one included) have international card acceptance disabled by default, so it fails with `international_transaction_not_allowed`. The Mastercard number above is domestic and works reliably.

7. Known Limitations

- **Admin/B2B "Forgot Password" pages are UI shells only** — the forms render but aren't wired to any API call yet (Customer login has no password, so this doesn't apply there).
- **SMS OTP delivery is not wired up** — OTPs are logged to the server console in development, and `123456` is always accepted as a fallback until a real SMS provider (e.g. MSG91, Twilio) is integrated.
- **The home page's decorative "Shop by category" tiles are static content** (curated imagery in a fixed 3-tile layout), separate from the real, admin-managed Category system used everywhere else (product forms, filters, B2B catalog).
- **No email notifications** for order confirmation, quote approval/rejection, etc. — all of that is visible in-app only for now.